

FACULTY OF ENGINEERING
UNIVERSITY OF PERADENIYA

Sri Lanka

Project Report

IoT-Based Predictive Maintenance System

Using Vibration Analysis on Electric Motors

Student IDs: E22215, E22028, E22177, E22287, E22199

Department: Electrical & Electronic Engineering

Venue: EngEX Exhibition, University of Peradeniya

Submitted in partial fulfilment of the requirements

Abstract

This report presents the design, implementation, and evaluation of a low-cost IoT-based predictive maintenance system for electric motors using vibration analysis. The system continuously acquires triaxial vibration data from an MPU6050 MEMS accelerometer interfaced with an ESP32 microcontroller. Root Mean Square (RMS) analysis is performed in real time to detect early mechanical anomalies including bearing wear, shaft misalignment, rotor imbalance, and mechanical looseness.

A laptop-based Python dashboard provides live visualization of vibration waveforms, RMS trend plots, and threshold-based fault alerts transmitted over Wi-Fi using the MQTT protocol. The prototype was validated by injecting controlled faults into a DC motor test rig, demonstrating distinct and repeatable changes in vibration signatures.

Results confirm that the system reliably distinguishes healthy motor operation from faulty conditions. The total bill of materials cost is approximately LKR 3,576 (under USD 10), demonstrating that effective predictive maintenance capability is achievable at a fraction of the cost of commercial monitoring solutions.

Keywords: Predictive Maintenance, Vibration Analysis, MPU6050, ESP32, IoT, MQTT, RMS, Condition Monitoring.

Contents

Abstract	i
1 Introduction	1
1.1 Motor Fault Classification	1
1.2 Maintenance Strategies	1
1.3 Motivation and Objectives	2
2 Literature Review	3
2.1 Vibration Analysis Fundamentals	3
2.2 RMS as a Fault Indicator	3
2.3 IoT-Based Condition Monitoring	4
2.4 Sensor Fusion	4
3 System Architecture	5
3.1 Overall Architecture	5
3.2 Communication Protocol	5
3.3 System Block Diagram	6
4 Hardware Design	7
4.1 Component Selection	7
4.2 MPU6050 Accelerometer	7
4.3 ESP32 Microcontroller	8
4.4 Circuit Description	8
4.5 Mechanical Sensor Mounting	8
5 Software Design	9
5.1 ESP32 Firmware	9
5.1.1 Task 1: Sensor Acquisition (Core 0)	9
5.1.2 Task 2: Processing and Communication (Core 1)	9
5.1.3 Threshold Adjustment	9
5.2 Python Dashboard	9
5.3 Fault Injection Mechanism	10

6	Signal Processing & Fault Detection	11
6.1	Root Mean Square Analysis	11
6.2	Window Parameters	11
6.3	Threshold Setting	11
6.4	Fault Signatures	12
6.5	Limitations of RMS-Only Detection	12
7	Dashboard & Visualisation	13
7.1	Dashboard Layout	13
7.2	Alert Mechanism	13
7.3	Data Logging	13
8	Experimental Setup & Testing	14
8.1	Test Rig Description	14
8.2	Test Procedure	14
8.3	Sensor Calibration	14
9	Results & Discussion	15
9.1	Baseline Performance	15
9.2	Imbalance Fault Results	15
9.3	Bearing Fault Results	15
9.4	Misalignment Fault Results	15
9.5	Summary of Results	16
9.6	System Latency	16
9.7	Discussion	16
10	Cost Analysis	17
11	Conclusion	19
11.1	Future Work	19

List of Tables

3.1	System layer overview	6
4.1	Component selection summary	7
6.1	Fault signatures and expected vibration response	12
9.1	RMS measurement results under different fault conditions (threshold = 0.20 g)	16
10.1	Bill of Materials with cost breakdown	18

1. Introduction

Electric motors are the backbone of modern industrial processes, driving compressors, pumps, fans, conveyors, and countless other critical machines. Their reliable and uninterrupted operation is essential to maintaining production efficiency and avoiding costly unplanned downtime. However, motors are subject to a variety of mechanical and electrical faults that, if left undetected, can escalate into catastrophic failures.

1.1 Motor Fault Classification

Motor faults are broadly classified into three categories:

- **Electrical faults:** insulation breakdown, stator winding failures, turn-to-turn shorts, overcurrent, and voltage imbalance.
- **Mechanical faults:** bearing wear, rotor imbalance, shaft misalignment, mechanical looseness, and rotor eccentricity.
- **Environmental faults:** overheating from blocked ventilation, contamination by dust or chemicals, and corrosion from moisture ingress.

Each fault type originates from a distinct physical degradation mechanism and manifests in characteristic changes in the motor's vibration, current, and temperature signatures.

1.2 Maintenance Strategies

Maintenance approaches for industrial rotating machinery are broadly classified as follows:

Reactive Maintenance Addresses failures after they occur. Results in unplanned downtime, potential secondary damage, and high emergency repair costs.

Preventive Maintenance Schedules inspections and part replacements at fixed time intervals. Often leads to unnecessary work on healthy components, increasing costs without proportional benefit.

Predictive Maintenance (PdM) Continuously monitors machine health indicators and triggers maintenance only when early warning signs of degradation are detected. Offers the best balance of reliability, safety, and cost-effectiveness.

1.3 Motivation and Objectives

This project addresses the challenge of making predictive maintenance accessible to small and medium-scale industries in Sri Lanka by developing a low-cost, ESP32-based IoT system for real-time vibration monitoring.

The primary objectives are:

1. Design and build a triaxial vibration acquisition node using the MPU6050 MEMS accelerometer and ESP32 microcontroller.
2. Implement on-device RMS computation and threshold-based fault detection in firmware.
3. Transmit sensor data wirelessly over Wi-Fi using the MQTT protocol to a laptop-based dashboard.
4. Develop a Python monitoring dashboard with live trend visualization and fault alerting.
5. Validate the system by injecting controlled mechanical faults into a DC motor test rig.
6. Demonstrate the system at the EngEX exhibition, University of Peradeniya.

The project was developed as part of the EN2160 Electronic Design Realization module by students E22215, E22028, E22177, E22287, and E22199 of the Faculty of Engineering, University of Peradeniya.

2. Literature Review

Vibration analysis is one of the most mature and widely used condition monitoring techniques for rotating machinery. The fundamental principle is that mechanical faults generate specific vibration signatures that can be captured by accelerometers and analysed to infer machine health.

2.1 Vibration Analysis Fundamentals

The standard workflow for vibration-based fault detection involves mounting a calibrated accelerometer on the machine housing, digitising the analog voltage signal at a sufficient sampling rate (typically 1–5 kHz for motor condition monitoring), and applying signal processing algorithms to extract fault-sensitive features.

The Fast Fourier Transform (FFT) converts time-domain signals to the frequency domain, where individual fault frequencies can be identified. For rolling-element bearings, characteristic defect frequencies are related to the ball-pass outer-race rate (BPOR), ball-pass inner-race rate (BPIR), and ball-spin frequency (BSF), all of which are functions of the bearing geometry and shaft rotational speed.

ISO 10816 provides internationally recognised vibration severity charts that classify machine condition as *Good*, *Satisfactory*, *Unsatisfactory*, or *Unacceptable* based on vibration velocity RMS values.

2.2 RMS as a Fault Indicator

The Root Mean Square (RMS) value of a vibration signal represents the overall energy content and is one of the most widely used scalar health indicators in industrial practice. An increase in broadband RMS is associated with the progressive development of mechanical faults. While spectral analysis provides deeper insight into fault type, RMS-based thresholding is practical for real-time embedded implementation due to its simplicity and low computational cost.

2.3 IoT-Based Condition Monitoring

Recent literature has demonstrated the feasibility of deploying low-cost MEMS accelerometers (MPU6050, ADXL345) on microcontroller platforms (Arduino, ESP32, Raspberry Pi) for motor condition monitoring. The ESP32 is particularly well-suited for this application due to its integrated 802.11 Wi-Fi, dual-core processor at 240 MHz, and hardware I2C/SPI peripherals.

Studies such as Rakesh et al. [1] have shown that machine learning classifiers trained on vibration features (RMS, kurtosis, crest factor) can achieve high classification accuracy in distinguishing healthy motors from those with bearing faults, imbalance, and misalignment.

2.4 Sensor Fusion

Sensor fusion – the intelligent combination of data from multiple sensors – can significantly enhance fault detection accuracy. Integrating vibration measurements with temperature and current data provides complementary diagnostic information: temperature rise correlates with bearing friction and overheating faults; current signatures carry information about rotor eccentricity and stator winding asymmetry. The present project focuses on vibration as the primary sensing modality, with provisions for adding current sensing as a future enhancement.

3. System Architecture

The system is organised into three functional layers: the sensing and data acquisition layer, the edge processing layer, and the monitoring and visualisation layer.

3.1 Overall Architecture

The MPU6050 triaxial MEMS accelerometer is mounted on the motor housing and communicates with the ESP32 over the I2C bus at 400 kHz. The ESP32 performs accelerometer data acquisition, computes RMS values from windowed vibration buffers, and transmits JSON data packets to the laptop over Wi-Fi using the MQTT protocol. The laptop-based Python dashboard subscribes to the MQTT broker, logs data, displays live waveforms and RMS trends, and triggers visual and audible alerts when RMS thresholds are exceeded.

The data flow is:

1. MPU6050 samples acceleration at 1 kHz on three axes (X, Y, Z).
2. ESP32 buffers 256 samples per window and computes per-axis RMS.
3. RMS values and metadata are packaged as a JSON payload and published via MQTT.
4. Python dashboard subscribes, parses, plots, and evaluates alert conditions.
5. Buzzer and LED indicators on the hardware node provide local fault signalling.

3.2 Communication Protocol

MQTT (Message Queuing Telemetry Transport) was selected for its lightweight publish-subscribe architecture, which is well-suited to resource-constrained IoT devices. The ESP32 acts as an MQTT client publishing to topic `motor/vibration`. The laptop runs a Mosquitto MQTT broker and a Python subscriber. For the EngEX demonstration, all devices operate on a local Wi-Fi hotspot.

3.3 System Block Diagram

Sensing Layer	Edge Processing Layer	Monitoring Layer
MPU6050 (I2C)	ESP32 Microcontroller	Laptop Dashboard
Triaxial Acceleration	RMS Computation	Python + matplotlib
1 kHz Sampling	MQTT Publish	Real-time Visualisation
	Buzzer + LED Alerts	CSV Data Logging

Table 3.1: System layer overview

4. Hardware Design

4.1 Component Selection

Component selection was guided by the objectives of low cost, availability in the Sri Lankan market, and suitability for exhibition demonstration. Table 4.1 lists the selected components.

Component	Model / Specification	Purpose
MPU6050	InvenSense 6-DOF IMU	Triaxial acceleration measurement
ESP32	Dev Kit v1 (Espressif)	Data acquisition, Wi-Fi, MQTT client
DC Motor	12 V, 300 RPM	Fault injection test rig
Battery	3.7 V Li-ion 2000 mAh	Portable standalone operation
Status LEDs	Red / Green / Yellow 5 mm	Local fault indication
Buzzer	5 V Passive	Audible fault alert
Push Buttons	Momentary SPST $\times 2$	Threshold calibration / reset
Potentiometer	10 k Ω rotary	RMS threshold adjustment
Voltage Divider	10 k Ω resistors	ESP32 power conditioning

Table 4.1: Component selection summary

4.2 MPU6050 Accelerometer

The MPU6050 is a 6-degree-of-freedom InvenSense IMU combining a triaxial accelerometer and triaxial gyroscope on a single chip. For this application only the accelerometer

axes are used. Key specifications relevant to motor condition monitoring are:

- **Measurement range:** $\pm 2g$, $\pm 4g$, $\pm 8g$, or $\pm 16g$ (configured to $\pm 8g$).
- **Resolution:** 16-bit ADC providing 4096 LSB/g at $\pm 8g$ range.
- **Sampling rate:** Configurable up to 1 kHz via Digital Low-Pass Filter (DLPF) registers.
- **Interface:** I2C up to 400 kHz fast mode.
- **Supply voltage:** 3.3 V, compatible with ESP32 GPIO.

4.3 ESP32 Microcontroller

The ESP32 (Espressif Systems) features a dual-core Xtensa LX6 processor at 240 MHz, 520 KB SRAM, integrated 802.11 b/g/n Wi-Fi, and Bluetooth. Its hardware I2C peripheral communicates with the MPU6050 at 400 kHz. One core handles sensor data acquisition and RMS computation while the second core manages the Wi-Fi stack and MQTT publishing, ensuring communication latency does not compromise sampling timing.

4.4 Circuit Description

The MPU6050 SDA and SCL lines connect to ESP32 GPIO21 and GPIO22 respectively, with 4.7 k Ω pull-up resistors to 3.3 V. The AD0 pin is tied to GND to set the I2C address to 0x68. The ESP32 is powered from the Li-ion battery through a 3.3 V LDO regulator. LED indicators are driven through 330 Ω current-limiting resistors. The buzzer is driven via a 2N2222 NPN transistor switch with a 1 k Ω base resistor.

4.5 Mechanical Sensor Mounting

The MPU6050 module is mounted on the motor housing using a custom 3D-printed bracket, ensuring rigid coupling and minimising sensor-to-structure compliance. The sensor is oriented such that:

- Z-axis is aligned with the motor shaft axis (axial direction).
- X-axis is radial (perpendicular to shaft, vertical).
- Y-axis is tangential (perpendicular to shaft, horizontal).

This orientation maximises sensitivity to bearing-related radial vibrations on the X-axis.

5. Software Design

5.1 ESP32 Firmware

The firmware is developed using the Arduino framework in PlatformIO. The architecture follows a task-based design using FreeRTOS with two independent tasks running on separate processor cores.

5.1.1 Task 1: Sensor Acquisition (Core 0)

- Initialises MPU6050 via I2C; configures accelerometer range to $\pm 8g$ and DLPF bandwidth to 94 Hz.
- Reads 256 consecutive accelerometer samples from all three axes at 1 kHz intervals using a hardware timer interrupt.
- Buffers samples in a circular FIFO; signals the processing task via a FreeRTOS semaphore when a full window is available.

5.1.2 Task 2: Processing and Communication (Core 1)

- On semaphore release, computes per-axis RMS from the 256-sample window.
- Evaluates RMS against user-adjustable threshold.
- Activates green LED (normal) or red LED and buzzer (fault).
- Serialises data as JSON: `{"ax_rms": 0.12, "ay_rms": 0.08, "az_rms": 0.11, "ts": 12345}`.
- Publishes JSON payload to MQTT broker on topic `motor/vibration` every 500 ms.

5.1.3 Threshold Adjustment

The fault detection threshold is mapped from the potentiometer analog read voltage (0–3.3 V) to an RMS threshold range of 0.1–2.0 *g*, allowing real-time sensitivity adjustment during demonstration.

5.2 Python Dashboard

The laptop-side monitoring application is implemented in Python 3 using `paho-mqtt`, `matplotlib`, and `numpy`. The processing loop is:

1. Subscribe to the MQTT broker on topic `motor/vibration`.
2. Parse incoming JSON payloads and append RMS values to rolling deque buffers (500 samples per axis).
3. Every 200 ms, refresh three subplots showing X, Y, and Z axis RMS trends with horizontal threshold reference lines.
4. Display a status panel with current and maximum RMS values and a colour-coded fault indicator.
5. Log all data to a timestamped CSV file for post-analysis.

5.3 Fault Injection Mechanism

To simulate motor faults during the EngEX demonstration, the test rig supports four controllable fault modes:

Imbalance A 5 g eccentric mass bolted to the shaft coupling at a fixed angular position, creating a once-per-revolution centrifugal force.

Misalignment Motor-to-load shaft coupling deliberately offset by approximately 1 mm laterally.

Looseness Mounting bolts slightly loosened to introduce structural compliance.

Bearing Wear Artificially worn bearing with surface defects substituted for the baseline bearing.

6. Signal Processing & Fault Detection

6.1 Root Mean Square Analysis

The Root Mean Square of a discrete vibration signal $x[n]$ over a window of N samples is defined as:

$$\text{RMS} = \sqrt{\frac{1}{N} \sum_{n=0}^{N-1} x[n]^2} \quad (6.1)$$

This quantity represents the square root of the mean power of the signal and is proportional to the overall vibration energy level. A healthy motor exhibits a characteristic baseline RMS value that remains approximately constant over time. Progressive mechanical degradation causes the RMS to increase monotonically above the baseline, making RMS trending a reliable early warning indicator.

6.2 Window Parameters

A window size of $N = 256$ samples was selected as a compromise between temporal resolution and RMS estimation accuracy. At a sampling rate of $f_s = 1$ kHz, each window covers:

$$T_w = \frac{N}{f_s} = \frac{256}{1000} = 256 \text{ ms} \quad (6.2)$$

This duration is sufficient to capture multiple vibration cycles at typical operating speeds (500–3000 RPM) while providing an RMS update rate of approximately 4 Hz, which is adequate for trending purposes.

6.3 Threshold Setting

The fault detection threshold τ is set empirically by observing the baseline RMS during a 30-second healthy operation period and multiplying by a safety factor of 1.5:

$$\tau = 1.5 \times \overline{\text{RMS}}_{\text{baseline}} \quad (6.3)$$

This conservative multiplier reduces false positives while remaining sensitive to early-stage fault development. The threshold is adjustable via the onboard potentiometer and is displayed on the dashboard.

6.4 Fault Signatures

Table 6.1 summarises the expected vibration response for each fault type under investigation.

Fault Type	Effect on RMS	Dominant Axis	Alert
Healthy Baseline	Stable, low RMS (~0.05–0.15 <i>g</i>)	All similar	None
Rotor Imbalance	Elevated radial at 1× RPM	X (radial)	Red
Shaft Misalignment	Elevated axial & radial at 1×, 2× RPM	Z + X	Red
Bearing Wear	Broadband increase; impulse transients	X and Y	Red
Mechanical Looseness	Increase with sub-harmonic content	All axes	Red

Table 6.1: Fault signatures and expected vibration response

6.5 Limitations of RMS-Only Detection

While RMS-based anomaly detection is effective for detecting the presence and severity of mechanical degradation, it does not inherently identify the specific fault type. Distinguishing between imbalance, misalignment, and bearing faults requires spectral analysis (FFT) or more sophisticated time-frequency methods. Future work will integrate FFT-based analysis on the laptop dashboard.

7. Dashboard & Visualisation

7.1 Dashboard Layout

The Python monitoring dashboard is designed for clarity and ease of interpretation during exhibition demonstrations. The interface is divided into:

- **Live RMS Trend Plots (3 subplots):** Rolling 500-sample history of X, Y, Z axis RMS with threshold reference lines.
- **Status Panel:** Current and maximum RMS values for all axes, system uptime, and a colour-coded fault status indicator.
- **Raw Waveform View (optional tab):** Real-time display of the most recent 256-sample raw acceleration buffer, useful for observing impulse transients.
- **Data Log:** Scrollable table of the last 50 measurements with timestamps and fault status.

7.2 Alert Mechanism

When any axis RMS exceeds the threshold, the dashboard triggers:

1. Visual colour change of status panel background from green to red.
2. Pop-up alert message identifying the faulted axis and measured RMS value.
3. Log entry with timestamp, RMS values, and fault status annotation.
4. Audible system notification on the laptop.

Simultaneously, the ESP32 hardware activates the red LED and buzzer at the sensor node, providing a local physical indication.

7.3 Data Logging

All incoming MQTT messages are logged to a timestamped CSV file (`vibration_log_YYYYMMDD_HHMMSS`) containing columns for timestamp, `ax_rms`, `ay_rms`, `az_rms`, threshold, and fault status.

This log enables post-hoc analysis and reporting of fault events.

8. Experimental Setup & Testing

8.1 Test Rig Description

The test rig consists of a small DC motor (rated 12 V, 300 RPM) mounted on an aluminium base plate. The motor shaft is coupled to a free-spinning load shaft supported by two ball bearings in pillow-block housings. The MPU6050 sensor module is bolted to the motor housing on the drive-end side. A variable DC power supply controls the motor speed to allow testing at multiple operating points.

8.2 Test Procedure

Testing was conducted in four sequential phases:

1. **Baseline Calibration:** Motor operated at nominal speed (200 RPM) for 30 seconds with no artificial faults. RMS values recorded to establish baseline statistics and set detection thresholds.
2. **Imbalance Fault Test:** 5g eccentric mass attached to shaft coupling. Motor operated for 60 seconds. RMS increase and alert activation recorded.
3. **Bearing Fault Test:** Original bearing replaced with defective bearing. Motor operated for 60 seconds. Broadband RMS increase and impulse transients observed.
4. **Recovery Test:** Faults removed, motor restored to baseline condition. System verified to de-assert fault alerts within one measurement window of fault removal.

8.3 Sensor Calibration

The MPU6050 was calibrated by placing the sensor on a stable, level surface and averaging 1000 accelerometer readings at rest to determine the offset and scale factor for each axis. These calibration coefficients are stored in ESP32 flash memory and applied to all subsequent measurements to remove DC bias and gravity contributions.

9. Results & Discussion

9.1 Baseline Performance

During healthy motor operation at 200 RPM, the measured RMS values were approximately 0.09 g (X-axis), 0.07 g (Y-axis), and 0.06 g (Z-axis). These values were stable within ± 0.01 g over a 5-minute monitoring period, confirming that the sensor and firmware configuration provide a reliable baseline measurement. The detection threshold was set to 0.20 g (approximately $2\times$ the peak baseline RMS).

9.2 Imbalance Fault Results

With the eccentric mass attached, the X-axis RMS increased to 0.38 g – more than double the threshold – and the fault alert was activated within one measurement window (256 ms) of fault injection. The Z-axis showed a smaller increase (0.14 g), consistent with the radial-dominant signature expected for rotor imbalance.

9.3 Bearing Fault Results

The defective bearing produced an X-axis RMS of 0.31 g and a Y-axis RMS of 0.29 g . The raw waveform view showed characteristic impact transients superimposed on the baseline vibration – a hallmark of bearing defect-generated impulse forces. The RMS threshold was exceeded consistently, and no false recovery events were observed during the 60-second test.

9.4 Misalignment Fault Results

With deliberate lateral shaft offset, the Z-axis (axial) RMS increased to 0.22 g and the X-axis (radial) RMS to 0.25 g , consistent with the elevated axial and radial vibration expected for angular misalignment. The fault alert was correctly activated within the first measurement window.

9.5 Summary of Results

Test Condition	X RMS (<i>g</i>)	Y RMS (<i>g</i>)	Z RMS (<i>g</i>)	Fault Alert
Healthy Baseline	0.09	0.07	0.06	None
Imbalance Fault	0.38	0.12	0.14	ACTIVATED
Bearing Fault	0.31	0.29	0.08	ACTIVATED
Misalignment Fault	0.25	0.11	0.22	ACTIVATED
After Fault Removal	0.10	0.08	0.06	None

Table 9.1: RMS measurement results under different fault conditions (threshold = 0.20 *g*)

9.6 System Latency

End-to-end latency from fault occurrence to dashboard alert was measured as approximately 600–900 ms, comprising:

- 256 ms for the acquisition window.
- ≈ 10 ms for ESP32 RMS computation and MQTT publishing.
- 300–600 ms for Wi-Fi transmission and Python callback processing.

This latency is well within acceptable limits for predictive maintenance applications, where the time scale of mechanical degradation is minutes to hours.

9.7 Discussion

The results demonstrate that the system successfully achieves its core objective of detecting simulated motor faults using vibration-based RMS analysis. The clear separation between healthy and faulty RMS values confirms that the chosen threshold ($2\times$ baseline) provides adequate sensitivity and specificity for the fault magnitudes tested.

The primary limitation of the current implementation is the use of scalar RMS as the sole feature, which can detect the presence and severity of a fault but cannot differentiate between fault types. Future enhancements including FFT-based spectral analysis and machine learning classifiers would enable fault classification.

10. Cost Analysis

One of the primary objectives was to demonstrate that effective predictive maintenance can be achieved at minimal cost. Table [10.1](#) presents the full Bill of Materials (BOM) for the prototype.

Component	Model	Qty	Unit (LKR)	Total (LKR)
Vibration Sensor	MPU6050 Module	1	200.00	200.00
Current Sensor	ACS712 Module	1	170.00	170.00
Microcontroller	ESP32 Dev Kit	1	1,650.00	1,650.00
DC Motor + Bearing	12 V DC Motor	4	55.00	220.00
Battery Pack	3.7 V 2000 mAh	1	750.00	750.00
LEDs (R/G/Y)	5 mm LEDs	1 set	4.25	12.75
Buzzer	5 V Passive	1	55.00	55.00
Potentiometer	10 k Ω	1	20.00	20.00
Push Buttons	Momentary SPST	2	85.00	170.00
Voltage Controller	10 k Ω divider	1	75.00	75.00
Connecting Wires	Jumper set 1 m	1	23.00	23.00
PCB/Breadboard	Half-size	1	150.00	150.00
Resistors/Caps	Assorted	1 pk	80.00	80.00
			Total	3,575.75

Table 10.1: Bill of Materials with cost breakdown

The total BOM cost of approximately LKR 3,576 (under USD 10) compares favourably with commercial vibration monitoring systems such as SKF Multilog or Emerson AMS 2140, which typically cost USD 5,000–20,000 per monitoring point. This demonstrates that effective single-point vibration monitoring capability is achievable using commodity IoT components.

11. Conclusion

This project successfully developed and demonstrated a low-cost IoT-based predictive maintenance system for electric motors using vibration analysis. The system integrates an MPU6050 triaxial MEMS accelerometer, an ESP32 microcontroller, MQTT wireless communication, and a Python-based laptop dashboard to provide real-time vibration monitoring and threshold-based fault detection.

RMS-based anomaly detection was implemented in firmware and validated against three controlled fault conditions – rotor imbalance, bearing wear, and shaft misalignment – all of which were reliably detected within a single 256 ms measurement window. The system achieves end-to-end fault alert latency of under 1 second and operates entirely on a local Wi-Fi network without cloud infrastructure.

Key conclusions:

- Vibration-based predictive maintenance is technically feasible using commodity IoT hardware.
- RMS analysis provides a reliable, computationally efficient fault severity indicator for real-time embedded implementation.
- MQTT over Wi-Fi is a practical and low-latency communication protocol for industrial IoT condition monitoring.
- Controlled fault injection is an effective method for validating PdM system performance in a laboratory setting.
- The prototype cost of LKR 3,576 (under USD 10) demonstrates affordability for small and medium industries.

11.1 Future Work

Several enhancements are identified for future development:

- FFT spectral analysis and fault classification to distinguish between imbalance, misalignment, and bearing faults.
- Machine learning models (SVM, Random Forest) trained on multi-feature vibration datasets for automated fault diagnosis.

- Multi-node scalability: a single MQTT broker supporting simultaneous monitoring of multiple motor nodes.
- Cloud integration (AWS IoT Core or ThingSpeak) for remote monitoring and historical trend storage.
- Sensor fusion with current and temperature measurements for comprehensive motor health assessment.
- Mobile application for field engineers to receive push notifications and view remote trend data.

Bibliography

- [1] A. Rakesh, T. T. Sreenu, S. K. Hasane, S. D. Pande, N. Jindal, and J. Singla, “Fault identification of induction motor using machine learning,” in *Proc. 9th Int. Conf. Inventive Systems and Control (ICISC 2025)*, IEEE Xplore, 2025.
- [2] R. Salunda, A. Štefková, and R. Langer, *Easy Basics of Vibration Diagnostics: Beginner’s Guide*, Adash Ltd., Czech Republic.
- [3] *ISO 10816-1:1995, Mechanical Vibration – Evaluation of Machine Vibration by Measurements on Non-Rotating Parts*, International Organization for Standardization, Geneva, Switzerland, 1995.
- [4] InvenSense Inc., *MPU-6000 and MPU-6050 Product Specification Revision 3.4*, Document Number PS-MPU-6000A-00, 2013.
- [5] Espressif Systems, *ESP32 Technical Reference Manual, Version 5.1*, 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf
- [6] “Vibration analysis basics,” YouTube. [Online]. Available: https://www.youtube.com/watch?v=2_o1SDy6__U
- [7] “Vibration monitoring and fault diagnosis,” YouTube. [Online]. Available: https://www.youtube.com/watch?v=BPMjYJ_HoWk
- [8] “Machine vibration analysis playlist,” YouTube. [Online]. Available: <https://www.youtube.com/watch?v=qcHjDLCJxfI>
- [9] Eclipse Foundation, “Eclipse Mosquitto MQTT Broker,” Version 2.0. [Online]. Available: <https://mosquitto.org>
- [10] “Predictive Maintenance Explained,” HEALPARS. [Online]. Available: <https://www.healpars.com/predictive-maintenance-explained>